

Metodología de laboratorio y room

Objetivos

Pasar de evidencia a hipótesis comprobable, diseñar pruebas mínimas, reconocer niveles de confirmación y abandonar líneas sin soporte.

Flujo general

1. Confirma alcance, objetivo, restricciones y horario.
2. Crea inventario de hosts, nombres, puertos, servicios, tecnologías e identidades.
3. Convierte cada observación en una hipótesis concreta.
4. Define la evidencia que confirmaría y la que refutaría.
5. Ejecuta la prueba mínima de menor riesgo.
6. Interpreta resultado y control negativo.
7. Amplía solo si cambia la conclusión o el impacto.
8. Registra evidencia y siguiente decisión.

Plantilla de hipótesis

Observación	Hipótesis	Prueba mínima	Resultado esperado	Resultado obtenido	Conclusión

Plantilla de entrada a sink

- Entrada controlada:
- Transformaciones conocidas:
- Componente que procesa la entrada:
- Sink potencial:
- Restricciones observadas:
- Primitiva que quiero demostrar:
- Evidencia esperada:
- Payload mínimo:
- Adaptaciones posibles:

Mapa de superficie

Organiza por frontera, no por herramienta:

Frontera	Activos	Entradas	Identidad	Dependencia	Evidencia
Internet-web	vhost/API	path, query, body, headers	anónima/usuario	DB, filesystem	petición-respuesta
Host local	procesos/servicios	config, entorno, archivos	usuario efectivo	kernel, sudo	permisos y ejecución
Dominio	cuentas/equipos	LDAP, SMB, Kerberos	principal	DC, AD CS	relaciones/tickets

Niveles de certeza

- No confirmada: posibilidad basada en diseño o tecnología.
- Indicio: comportamiento compatible, pero con explicaciones alternativas.
- Confirmación: prueba reproducible demuestra la primitiva.
- Explotación: se usa la primitiva para obtener una capacidad adicional.
- Impacto: consecuencia sobre confidencialidad, integridad o disponibilidad dentro del alcance.

No llames SQLi a un 500 ni SSRF a una demora aislada. Añade controles: entrada inocua, destino inexistente, repetición y comparación.

Cuando falla la prueba básica

Clasifica antes de adaptar:

1. transporte incorrecto;
2. entrada no controlada;
3. parser distinto;
4. sintaxis/contexto incorrectos;
5. normalización o filtro;
6. sink no alcanzado;
7. efecto sin canal observable;
8. hipótesis falsa.

Cada adaptación debe discriminar entre dos causas. Probar listas aleatorias aumenta ruido sin aumentar conocimiento.

Investigar herramientas y errores

Extrae nombre, versión, sintaxis mostrada y fase del error. Consulta primero manual oficial, ayuda local y documentación del proyecto. Reproduce la mínima invocación en un entorno propio. Un mensaje puede proceder de proxy, framework, librería, herramienta externa o sistema operativo; atribuirlo al componente equivocado produce payloads inútiles.

Abandonar una hipótesis

Detente cuando la entrada no llega al componente, controles positivos y negativos son indistinguibles, la evidencia contradice el mecanismo, o el coste/riesgo supera el valor. Registra qué se descartó y bajo qué condiciones; podría reabrirse con nueva evidencia.

Pistas progresivas

1. identificar la entrada;
2. nombrar el componente;
3. indicar la primitiva;
4. señalar la restricción;
5. mostrar un fragmento, nunca la solución completa antes del intento.

Caso completo: la demora engañosa

GET /report?name=test;sleep%205 tarda 5,8 segundos una vez. La hipótesis inicial es command injection blind.

1. Observación: una única petición fue lenta.
2. Qué sé realmente: solo existe correlación temporal en una muestra.

3. Hipótesis: H1, una shell ejecutó sleep; H2, el informe entró en una cola lenta; H3, el backend reintentó una dependencia.
4. Experimento discriminatorio: alternar diez pares aleatorios de control y payload, usar demoras de 2 y 5 segundos y registrar una cabecera de correlación.
5. Resultado esperado: H1 escala con el número solicitado y solo en el payload; H2/H3 produce colas o reintentos no proporcionales.
6. Resultado obtenido: controles y payloads forman la misma distribución; el log muestra tres reintentos HTTP.
7. Conclusión: la señal inicial era un falso positivo.
8. Siguiente paso: investigar la dependencia solo si está dentro del alcance; cerrar la hipótesis de shell.

Caso completo: dos hipótesis compatibles

Una URL de avatar aparece en el perfil y un listener externo recibe una petición. Puede ser el navegador del usuario o el servidor.

Experimento: guardar una URL única, cerrar el navegador y solicitar el perfil desde un cliente que no cargue imágenes; después comparar IP origen, User-Agent y tiempo. Si el callback ocurre durante el guardado desde la red del servidor, apoya petición server-side. Si solo ocurre al renderizar y tiene el navegador como origen, es carga client-side. El nombre del parámetro no decide la vulnerabilidad: la entidad que inicia la conexión sí.

Registro de decisión

Para cada intento añade una fila, incluso cuando falla:

Paso	Evidencia nueva	Hipótesis favorecida	Hipótesis debilitada	Próxima prueba y motivo
1				

Esta tabla evita convertir una cronología de payloads en una falsa explicación causal.

Referencias

- [OWASP Web Security Testing Guide](#)
- [Guías internas](#)

Anterior: [Flujo de datos](#). Siguiente: [Construcción de payloads](#).